```
»           }
        }
    }
```

Trigger the connection by referencing the status code. In an ideal world, we'd check this value to make sure the request succeeded, but that's out of this book's scope. For now, all we care about is exceptions.

ch10_63/android/app/src/main/java/com/masilotti ...
journal/viewmodels/NotificationTokenViewModel.kt

```
    // ...

    class NotificationTokenViewModel {
        suspend fun registerToken(token: String) = withContext(Dispatchers.IO) {
            // ...

            OutputStreamWriter(connection.outputStream).use { writer ->
                writer.write(body.toString())
            }

»           connection.responseCode
        }
    }
```

Wrap the entire function implementation in a `try`/`catch` block to catch any exceptions raised when making the network request.

ch10_64/android/app/src/main/java/com/masilotti ...
journal/viewmodels/NotificationTokenViewModel.kt

```
    // ...

    class NotificationTokenViewModel {
        suspend fun registerToken(token: String) = withContext(Dispatchers.IO) {
»           try {
                val url = URL("$baseURL/notification_tokens")
                // ...

                connection.responseCode
»           } catch (e: Exception) {
»               e.printStackTrace()
```